

Query Performance

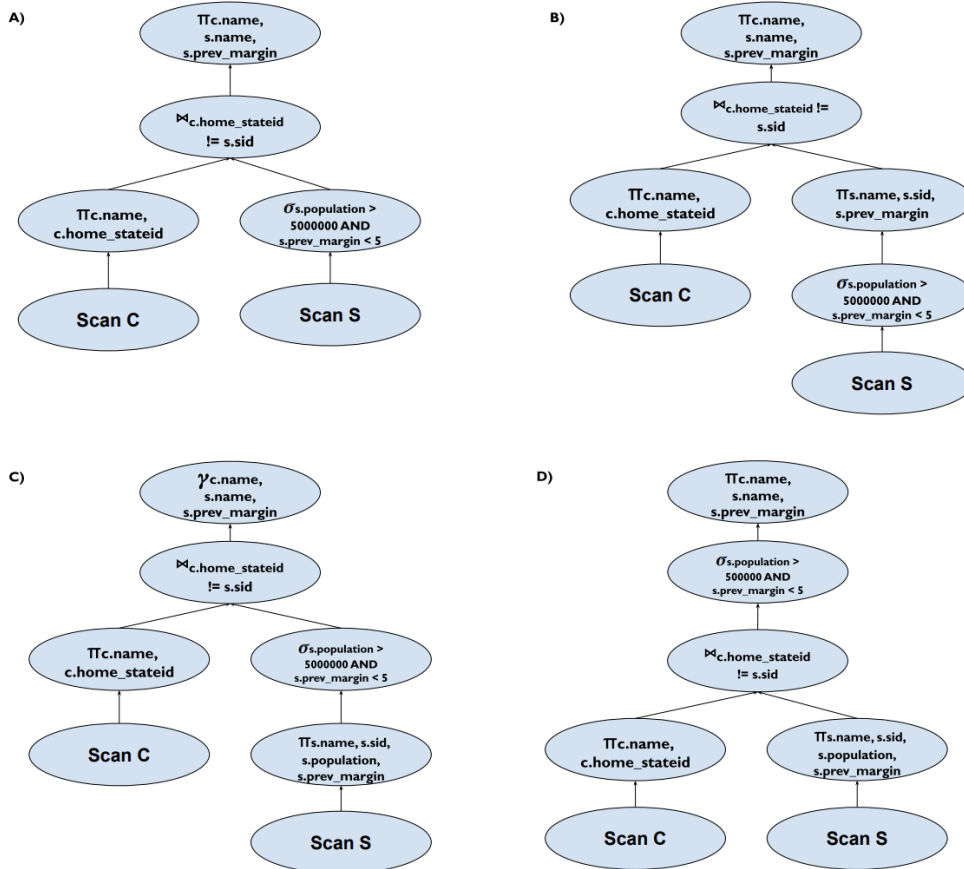
Instructor: Aditya Parameswaran

Covers Lecture(s): 7 – 8

I Query Optimization

1. Consider the following SQL query about electoral candidates. Which of the query plans is the optimized query plan for this query? What optimizations were used?

```
SELECT c.name, s.name, s.prev_margin
FROM candidates AS c INNER JOIN states AS s
ON c.home_stateid != s.sid
WHERE s.population > 5000000 AND s.prev_margin < 5;
```



Query plan B has all the optimizations applied. It has pushed down selections and projections.

Query plan A is not correct because the projection has not been pushed down to the right hand side of the join. The projection must be pushed down and applied to both tables prior to joining.

Query plan C is not correct because a GROUP BY/Aggregation is not necessary for the given SQL query.

Query plan D is not correct because the selection has not been pushed down.

II Join orders [Optional]

2. Use the following schema and information for this question.

- **students** (*student_id*, *name*, *age*, *major*) with 100 tuples
- **orders** (*student_id*, *book_id*, *date*) with 500 tuples
- **books** (*book_id*, *author*, *num_copies*) with 200 tuples

We want to do a natural join of the above three relations. What is the optimal join order?

A good heuristic in general is to join the relations with fewer tuples, but this might not always work! **students** and **books** each have the fewest number of tuples out of the three relations, but since they do not share a common column, a natural join between **students** and **books** would become a cross product, which means we would get $100 \times 200 = 20,000$ tuples out of the cross product. This is a large number of tuples which is unideal! However, if we first do a natural join between **students** and **orders** on the *student_id* column, we would get at most 500 matching tuples. We would then do a natural join between those tuples and the **books** relation on the *book_id* column. Since each order references at most one book, this final join produces at most 500 tuples as well, keeping intermediate and final result sizes small throughout.

III Query Plan Rewriting, Part I

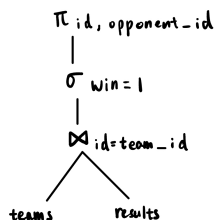
Consider the following schema:

- **teams** (*id*, *state*)
- **results** (*team_id*, *opponent_id*, *win*)

And the following SQL query retrieves a list of team IDs along with the IDs of opponents they defeated:

```
SELECT t.id, r.opponent_id
FROM teams AS t INNER JOIN results AS r
ON t.id = r.team_id
WHERE r.win = 1
```

3. *Logical Plan.* Write out the logical plan based on order of evaluation, prior to optimization. Write it as a tree of relational algebra operators. (Hint: The order of the operators should be join, filter, then projection.)



4. *Predicate Push down.* One way to optimize this query plan is to perform predicate push down, where we move the σ around. What would the logical plan look like if we push the predicate through the join?



5. Why may we want to use the second query plan instead of the first?

We might prefer the second query plan because pushing the predicate (the filtering condition) down before the join reduces the number of rows that need to be processed in the join operation. This improves performance by ensuring that only the relevant rows (teams with wins) are included in the join, rather than considering all rows first and filtering afterward.

6. **Challenge:** Note that we have not placed any foreign key constraints. What is a situation in which the first query plan would perform better? *Hint:* Consider a scenario where the **results** table has invalid **team_id** values that don't match entries in the **teams** table.

Since there is no foreign key constraint on **team_id**, the results table might contain **team_id** values that don't exist in the teams table. If most result tuples don't match with team tuples (e.g., they have the results of matches from teams that are no longer active), the second query plan will require expensive filtering of all result tuples, while the first only needs to do so for result tuples that match. For the equi-join, the first query plan could also avoid scanning all the result tuples, e.g., by using an index.

Additionally, in an extreme case where there are no matching **results.team_id = teams.id** at all, the join would return an empty result set immediately. In this scenario, the join is highly efficient because the system would not need to apply the **win = 1** filter at all.

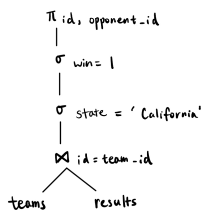
IV Query Plan Rewriting, Part II

Next, let's consider a somewhat more complicated query.

```
SELECT t.id, r.opponent_id
FROM teams AS t INNER JOIN results AS r
ON t.id = r.team_id
WHERE t.state = 'California' AND r.win = 1
```

Let's walk through the query optimization process again.

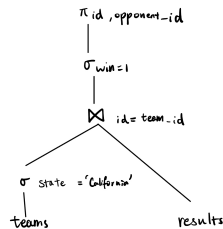
7. What does the logical query plan look like if we compute the join first, then filter by location, and then filter by wins?



8. Assume that after computing the joins, there are 50 rows that are in California, 100 rows that are wins, and 10 rows that are both. Without pushing predicates through the join, what is the optimal logical plan? (there are no indices available)

We want to first filter to California, then by wins. Regardless of the order, we must scan through all rows to evaluate the first predicate, but if we filter California first there are only 50 rows to scan through for the second predicate instead of 100.

9. Now assume there are 1000 teams total (10% of which are from California) and 5000 results total (half of which are wins). We have a hash index on both *teams* and *results* for the *id* and *team_id* columns, respectively. What is the optimal logical plan?



The crux of this problem lies in having hash indexes on the *id* and *team_id* columns of the *teams* and *results* tables, respectively. These indexes make the joining process very efficient, especially when one table is significantly smaller than the other. Given that only 10% of the 1,000 teams are from California, we can further reduce the size of the teams table to 100 rows by applying a filter early in the process.

Approximate lookups of two approaches:

(a) **Joining Before Filtering on win = 1:**

- 1,000 (scan `teams` to filter on `state='California'`, which left with 100 rows)
- + 100 (join using hash index, filter `win = 1`)
- **Total: ~1,100**

In practice, the predicate is usually included as part of the hash join with the index.

(b) **Filtering on win = 1 Before Joining:**

- 1,000 (scan `teams` to filter on `state='California'`, which left with 100 rows)
- + 5,000 (scan `results` to filter on `win=1`, which left with 2,500 rows)
- + 100 (join using hash index)
- **Total: ~6,100**

V EXPLAIN ANALYZE with Joins

Consider the following schema:

- **actors** (**id**, **name**)
- **cast_info** (**person_id**, **movie_id**)

10. Write a query that computes an inner join of **actors** and **cast_info** on the actor ID. Your query should return all attributes. What kind of join is performed? Why does the query optimizer choose this join method?

```
SELECT * FROM actors, cast_info WHERE actors.id = cast_info.person_id;
```

Hash join. Here are some reasons for using hash join:

- (a) Join condition: This is an equi-join, which is particularly suitable for hash join. If this were an inequality theta join, hash join would not be efficient here.
- (b) Table sizes: If one of the tables (**actors**) is substantially smaller than the other (**cast_info**), it can be loaded into memory and used to build a hash table, which can then be used for a more efficient join with the larger table. Here the smaller table actor (845,888 rows) is used to build the hash table. The larger table cast_info (2,211,936 rows) is then scanned and matched against this hash table.

11. Which settings are related to selecting the type of join that the query optimizer can select?

```
enable_hashjoin, enable_mergejoin, enable_nestloop
```

12. Let's turn off hash join as an option for the query optimizer. Rerun the same query in Question 10. Now what is the join the query optimizer picks? Why might this be preferred over the remaining alternatives?

Merge join. Here are some reasons for using merge join:

- (a) Join condition: merge join is the next most efficient join after turning off the hash join option.
- (b) Presence of Order and Indexes: There's an index on **actors.id** (as shown by the Index Scan using **actor_pkey**), which makes accessing rows from the actor table in sorted order efficient. This favors a merge join.
- (c) Table Sizes: Both **actors** and **cast_info** have a considerable number of rows. Nested loop joins tend to be efficient when one of the tables is significantly smaller, allowing for rapid repeated lookups or scans of that table. Given the sizes of the tables, a nested loop join might have been seen as less efficient than a merge join.

13. Write a SQL command that turns off the join method used in the previous question. What do you think the query optimizer will use now?

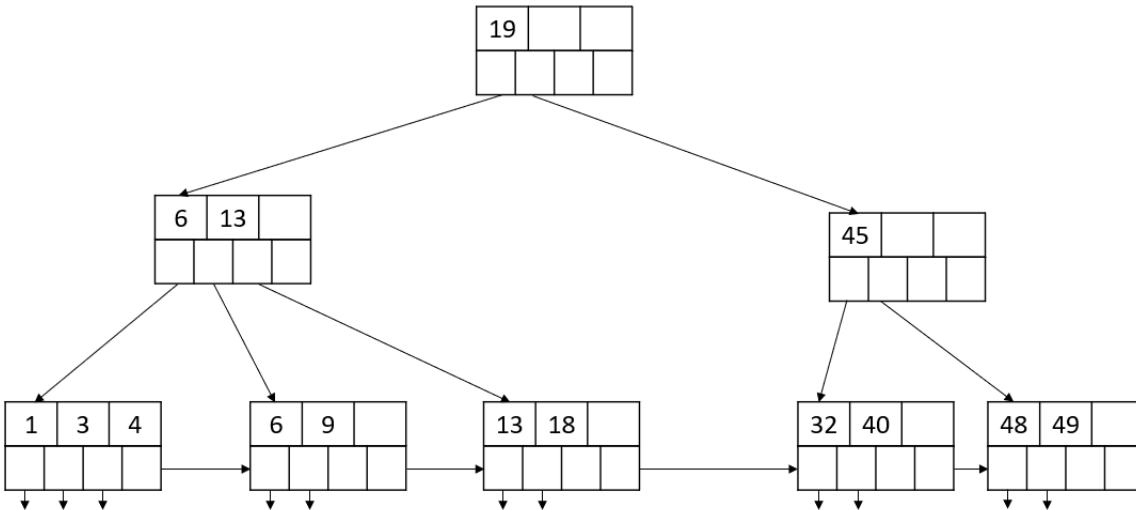
```
set enable_mergejoin=false;
```

The SQL engine can only use nested loop join. As you see, it is significantly slower.

14. Write two SQL commands that reenables the two join methods you disabled in previous questions.

```
set enable_hashjoin=true; set enable_mergejoin=true;
```

VI Indexes [Optional]



15. Describe how to navigate a B+ tree in general.

We start with the root node and execute a search similar to binary tree search. We follow the pointers that lead to key values that contain the key we're searching for in its range. This is repeated recursively until we reach the leaf level – if the goal key is in the leaf node, return the appropriate value; otherwise, return that the goal key does not exist. We don't stop searching until we reach the leaf node.

16. Describe the traversal route if we search for 48 in the given B+ tree above. How many nodes are reached?

We start at the root node, which contains the key 19. We see that our goal key 48 is greater than 19, so we traverse to the node right of 19. In this intermediate node, we see that it contains the key 45. We know that 48 is greater than 45 so we traverse to the node right of 45. We arrive at a leaf node. We see that the leaf node contains our goal key 48 so we return the corresponding value. We are done. Route: 19 → 45 → 48. We touch 3 nodes in total.

17. Describe how to do a range search on keys 5 through 20 inclusive in the given B+ tree above. How many nodes are reached?

Let the start of the range be our goal key, 5. Starting with the root node, we see that it contains the key 19. Since 5 is less than 19, we traverse the node left of 19. We arrive at the intermediate node and see that it contains the keys 6 and 13. Since 5 is less than 6, we traverse the node left of 6. We arrive at a leaf node that contains keys 1, 3, and 4. Since key 5 is not in this leaf node, we go to the sibling node. The sibling node contains keys 6 and 9 so we add the corresponding

values to our result. Since the largest key we've seen is 9, we continue to traverse the right siblings. The next right sibling contains keys 13 and 18, and we add the corresponding values to our result. Since 18 is less than our range max 20, we travel to the right sibling once more. We see that the smallest key is 32, which is outside of our desired range from 5 to 20 inclusive, so we end our range search here. We touch 6 nodes in total.